

Sí. Entonces el objetivo queda así:

```
\[ \boxed{\text{1 único dataset longitudinal}} \]
```

para el **party_id** que tú ya tienes, desde **2022 hasta su observation_date**, conservando **todos los rows originales de trades**.

Además, el mismo dataset tendrá:

- todas las columnas originales de **trades**;
- identificación **HISTORICAL / SEED / PRIOR_9M / RECENT_3M**;
- variables mensuales construidas durante los 12 meses oficiales;
- weighted utilization mensual;
- features finales únicamente en las filas del **observation_date**.

No usamos **DISTINCT**, **GROUP BY**, **LIMIT** ni seleccionamos una sola cuenta, por lo que **no eliminamos rows del histórico original**.

1. Define el party que ya tienes

```
library(sparklyr)
library(dplyr)
```

```
party_id_target <- "AQUI_TU_PARTY_ID"
```

```
party_id_sql <- gsub(
  "''",
  "''",
  party_id_target,
  fixed = TRUE
)
```

Primero recuperamos su observation date directamente del dataset final:

```
sdf_sql(
  sc,
  sprintf(
    "
    CREATE OR REPLACE TEMP VIEW selected_party_case AS

    SELECT
      party_id,
      observation_date,
      observation_year,
      label_3m

    FROM bu_frd.model_features_ab_final_2022_2025
```

```

WHERE CAST(party_id AS STRING) = '%s'
",
party_id_sql
)
)

```

Comprueba que sea el caso correcto:

```

selected_party <- sdf_sql(
  sc,
  "
  SELECT *
  FROM selected_party_case
  "
) %>%
  collect()

```

selected_party

En tu caso esperaríamos algo como:

party_id	observation_date	observation_year	label_3m
XXXXXXX	2024-xx-xx	2024	1

2. Dataset longitudinal completo

Este es el query principal.

```

party_history_complete <- sdf_sql(
  sc,
  "
  WITH raw_history AS (

    SELECT

      s.observation_date,
      s.observation_year,
      s.label_3m,

      last_day(t.data_date) AS history_month,

      CAST(
        months_between(
          last_day(s.observation_date),
          last_day(t.data_date)

```

```

        ) AS INT
    ) AS history_month_offset,

    t.*

FROM trades t

INNER JOIN selected_party_case s
    ON t.party_id = s.party_id

WHERE t.data_date >= to_date('2022-01-01')

    AND t.data_date <= s.observation_date
),

history_tagged AS (

    SELECT
        r.*,

        CASE

            WHEN history_month_offset > 12
                THEN 'HISTORICAL'

            WHEN history_month_offset = 12
                THEN 'SEED'

            WHEN history_month_offset BETWEEN 3 AND 11
                THEN 'PRIOR_9M'

            WHEN history_month_offset BETWEEN 0 AND 2
                THEN 'RECENT_3M'

            ELSE 'CHECK_DATE'

        END AS period_type

    FROM raw_history r
)

SELECT

    /* =====
    CASE / TIME INFORMATION
    ===== */

    h.observation_date,

```

h.observation_year,
h.label_3m,

h.history_month,
h.history_month_offset,
h.period_type,

/* =====
ALL ORIGINAL TRADE COLUMNS
===== */

h.party_id,
h.trades_seq_num,
h.acct_id,
h.acct_num,
h.acct_typ_cde,
h.agcy_id_cde,
h.bloon_payt_amt,
h.bloon_payt_due_dte,
h.crdtr_cls_cde,
h.defr_payt_strt_dte,
h.ecoa_cde,
h.end_dte,
h.erl_maxdlq_rte_dte,
h.erl_mxdlq_psdu_amt,
h.inp_seg_src_cde,
h.kob_posn_1_cde,
h.kob_posn_2_cde,
h.legis_prhib_cde,
h.max_delq_amt,
h.max_delq_dte,
h.max_delq_rte_cde,
h.mortg_id_num,
h.open_dte,
h.orgn_chrg_off_amt,
h.orgn_cint_nme,
h.payt_schd_mth_cnt,
h.porf_sale_ind_cde,
h.port_sale_nme,
h.porf_typ_cde,
h.rem_frst_rpt_dte,
h.rsub_base_cde,
h.rsub_sfx_cde,
h.scnd_agcy_acct_num,
h.terms_freq_cde,

h.acct_mop_cde,

h.affil_rem_cde,
h.clsd_dte,
h.cmpinc_rem_cde,
h.cmplrem_frsrpt_dte,

h.cr_lmt_amt,
h.curr_bal_amt,

h.eff_dte,
h.gnre_rem_cde,
h.gnrcrem_frsrpt_dte,
h.high_cr_amt,
h.paid_out_dte,
h.past_due_amt,
h.payt_amt,
h.payt_dte,
h.payt_due_amt,
h.payt_pttrn_txt,
h.payt_typ_cde,
h.rte_rem_cde,
h.rte_rem_frsrpt_dte,
h.acct_verf_cde,
h.acct_rte_prfl_cde,

h.end_dte_age,
h.rem_frst_rpt_dte_age,
h.open_dte_age,
h.defr_payt_strt_dte_age,
h.bloon_payt_strt_dte_age,
h.max_delq_dte_age,
h.erl_max_delq_rte_dte_age,
h.eff_dte_age,
h.cmpl_rem_frst_rpt_dte_age,
h.gnrc_rem_frst_rpt_dte_age,
h.rte_rem_frst_rpt_dte_age,
h.cls_dte_age,
h.paid_out_dte_age,
h.payt_dte_age,

h.frst_delq_dte,
h.frst_rpt_dte,
h.acat_upd_tsp,
h.frst_rpt_dte_age,
h.frst_delq_dte_age,
h.acat_upd_tsp_age,
h.acat_upd_tsp_age_dys,

h.data_date,

```
/* =====  
MONTHLY DERIVED VARIABLES  
These only exist inside the official 12-month feature window  
===== */
```

```
m.feature_month,  
m.feature_window,  
  
m.reported_accounts,  
m.active_accounts,  
m.rated_active_accounts,  
m.utilization_eligible_accounts,  
  
m.avg_balance_active,  
m.total_balance_active,  
  
m.avg_past_due_active,  
m.total_past_due_active,  
  
m.delinquent_active_accounts,  
m.delinquent_account_share,  
  
m.avg_utilization,  
  
m.deterioration_count,  
m.improvement_count,  
  
m.adverse_entry_count,  
m.adverse_resolved_count,  
  
m.became_unrated_count,  
m.left_unrated_count,  
  
m.max_dpd_jump,
```

```
/* =====  
MONTHLY WEIGHTED UTILIZATION  
===== */
```

```
u.weighted_utilization,
```

```
/* =====  
FINAL ENGINEERED FEATURES  
Populate ONLY at the observation month.
```

Since several trades may exist in observation month,
these values will repeat across those observation-month rows.

===== */

CASE

 WHEN h.history_month = last_day(h.observation_date)
 THEN f.deterioration_acceleration
END AS deterioration_acceleration,

CASE

 WHEN h.history_month = last_day(h.observation_date)
 THEN f.adverse_acceleration
END AS adverse_acceleration,

CASE

 WHEN h.history_month = last_day(h.observation_date)
 THEN f.max_dpd_jump_recent_3m
END AS max_dpd_jump_recent_3m,

/* Active accounts */

CASE

 WHEN h.history_month = last_day(h.observation_date)
 THEN f.active_accounts_change_vs_baseline
END AS active_accounts_change_vs_baseline,

CASE

 WHEN h.history_month = last_day(h.observation_date)
 THEN f.active_accounts_changed_flag
END AS active_accounts_changed_flag,

/* Balance */

CASE

 WHEN h.history_month = last_day(h.observation_date)
 THEN f.balance_change_model
END AS balance_change_model,

CASE

 WHEN h.history_month = last_day(h.observation_date)
 THEN f.log1p_abs_balance_change_model
END AS log1p_abs_balance_change_model,

CASE

 WHEN h.history_month = last_day(h.observation_date)
 THEN f.balance_change_available_flag

END AS balance_change_available_flag,

CASE

WHEN h.history_month = last_day(h.observation_date)

THEN f.balance_recent_months_available

END AS balance_recent_months_available,

CASE

WHEN h.history_month = last_day(h.observation_date)

THEN f.balance_prior_months_available

END AS balance_prior_months_available,

/* Past due */

CASE

WHEN h.history_month = last_day(h.observation_date)

THEN f.past_due_change_model

END AS past_due_change_model,

CASE

WHEN h.history_month = last_day(h.observation_date)

THEN f.past_due_change_available_flag

END AS past_due_change_available_flag,

CASE

WHEN h.history_month = last_day(h.observation_date)

THEN f.past_due_recent_months_available

END AS past_due_recent_months_available,

CASE

WHEN h.history_month = last_day(h.observation_date)

THEN f.past_due_prior_months_available

END AS past_due_prior_months_available,

/* Delinquency */

CASE

WHEN h.history_month = last_day(h.observation_date)

THEN f.delinquency_share_change_model

END AS delinquency_share_change_model,

CASE

WHEN h.history_month = last_day(h.observation_date)

THEN f.delinquency_change_available_flag

END AS delinquency_change_available_flag,

```
CASE
  WHEN h.history_month = last_day(h.observation_date)
  THEN f.delinquency_recent_months_available
END AS delinquency_recent_months_available,
```

```
CASE
  WHEN h.history_month = last_day(h.observation_date)
  THEN f.delinquency_prior_months_available
END AS delinquency_prior_months_available,
```

```
/* Utilization */
```

```
CASE
  WHEN h.history_month = last_day(h.observation_date)
  THEN f.avg_utilization_change_model
END AS avg_utilization_change_model,
```

```
CASE
  WHEN h.history_month = last_day(h.observation_date)
  THEN f.weighted_utilization_change_model
END AS weighted_utilization_change_model,
```

```
CASE
  WHEN h.history_month = last_day(h.observation_date)
  THEN f.utilization_change_available_flag
END AS utilization_change_available_flag,
```

```
CASE
  WHEN h.history_month = last_day(h.observation_date)
  THEN f.utilization_recent_months_available
END AS utilization_recent_months_available,
```

```
CASE
  WHEN h.history_month = last_day(h.observation_date)
  THEN f.utilization_prior_months_available
END AS utilization_prior_months_available,
```

```
/* Z-score challengers */
```

```
CASE
  WHEN h.history_month = last_day(h.observation_date)
  THEN f.active_accounts_z_model
END AS active_accounts_z_model,
```

```
CASE
  WHEN h.history_month = last_day(h.observation_date)
```

```
    THEN f.active_accounts_z_available_flag
END AS active_accounts_z_available_flag,
```

```
CASE
    WHEN h.history_month = last_day(h.observation_date)
    THEN f.avg_utilization_z_model
END AS avg_utilization_z_model,
```

```
CASE
    WHEN h.history_month = last_day(h.observation_date)
    THEN f.avg_utilization_z_available_flag
END AS avg_utilization_z_available_flag,
```

```
CASE
    WHEN h.history_month = last_day(h.observation_date)
    THEN f.weighted_utilization_z_model
END AS weighted_utilization_z_model,
```

```
CASE
    WHEN h.history_month = last_day(h.observation_date)
    THEN f.weighted_utilization_z_available_flag
END AS weighted_utilization_z_available_flag
```

```
FROM history_tagged h
```

```
/* =====
Official monthly behavior.
No match for HISTORICAL or SEED rows => derived values stay NULL.
===== */
```

```
LEFT JOIN bu_frd.model_party_behavior_monthly_2022_2025 m
```

```
    ON h.party_id = m.party_id
    AND h.observation_date = m.observation_date
    AND h.history_month = m.feature_month
```

```
/* =====
Monthly utilization
===== */
```

```
LEFT JOIN bu_frd.model_party_utilization_monthly_2022_2025 u
```

```
    ON h.party_id = u.party_id
    AND h.observation_date = u.observation_date
    AND h.history_month = u.feature_month
```

```

/* =====
Final engineered features.
Joined once at party-observation level.
CASE statements control where they appear.
===== */

LEFT JOIN bu_frd.model_features_ab_final_2022_2025 f

  ON h.party_id = f.party_id
  AND h.observation_date = f.observation_date

ORDER BY
  h.data_date,
  h.acct_id,
  h.trades_seq_num
"
)

```

3. Ahora sí: recuperar todos los rows

No uses `head()` ni `LIMIT`.

```
party_history_complete_df <- party_history_complete %>%
  collect()
```

```
party_history_complete_df
```

Eso trae todas las filas del party desde:

2022 – 01 – 01

hasta:

observation_date

4. Qué deberías observar

Supongamos que el caso tiene `observation_date = 2024-12-31`.

La parte histórica podría verse conceptualmente así:

data_date	period_type	acct_id	MOP	balance	active_accounts	avg_balance	final balance change
2022-01-31	HISTORICAL	A001	01	2,300	NULL	NULL	NULL
2022-01-31	HISTORICAL	A002	01	8,700	NULL	NULL	NULL
2022-02-28	HISTORICAL	A001	01	2,450	NULL	NULL	NULL
...	...	...	...	...	...	...	...
2023-12-31	SEED	A001	01	3,600	NULL	NULL	NULL
2024-01-31	PRIOR_9M	A001	01	3,800	5	10,900	NULL
2024-01-31	PRIOR_9M	A002	01	7,100	5	10,900	NULL
...	...	...	...	...	...	...	...
2024-10-31	RECENT_3M	A001	01	5,100	7	17,200	NULL
2024-11-30	RECENT_3M	A001	02	5,400	7	17,800	NULL
2024-12-31	RECENT_3M	A001	01	5,200	8	18,900	7,400
2024-12-31	RECENT_3M	A002	01	13,700	8	18,900	7,400

Los números son solo ilustrativos.

Observa dos cosas importantes.

En **2024-01**, si tienes varias cuentas:

active_accounts = 5

avg_balance_active = 10,900

se repetirá en las diferentes cuentas de ese mes porque esas variables pertenecen al nivel:

party + month

No al nivel account.

Y en 2024-12, las features finales como:

```
balance_change_model  
active_accounts_change_vs_baseline  
avg_utilization_z_model
```

se repetirán entre las cuentas del observation month porque pertenecen al nivel:

party + observation_date

Eso es **esperado y correcto para este pull**, porque tú quieres preservar todos los rows originales.

5. QA que recomiendo ejecutar inmediatamente

Después de crear el dataset, comprobaría que no hayamos multiplicado accidentalmente los rows por los joins.

Primero cuenta el raw histórico:

```
raw_count <- sdf_sql(  
  sc,  
  "  
  SELECT  
    COUNT(*) AS raw_rows  
  
  FROM trades t  
  
  INNER JOIN selected_party_case s  
    ON t.party_id = s.party_id  
  
  WHERE t.data_date >= to_date('2022-01-01')  
    AND t.data_date <= s.observation_date  
  "  
) %>%  
  collect()  
  
raw_count
```

Después:

```
final_count <- party_history_complete %>%  
  summarise(  
    final_rows = n()  
  ) %>%
```

`collect()`

`final_count`

Debemos tener:

`\[ \boxed{raw\_rows=final\_rows} \]`

Si por ejemplo:

`raw_rows = 1,284`

`final_rows = 1,284`

perfecto.

Si obtuviéramos:

`raw_rows = 1,284`

`final_rows = 2,568`

entonces algún join está multiplicando filas y **no deberíamos usar el pull todavía**.

Ese QA es especialmente importante porque tu requisito principal es conservar exactamente **todos los rows originales, ni menos ni más**.

Una única precaución antes de ejecutarlo: confirma que en `model_party_utilization_monthly_2022_2025` la columna se llama exactamente `weighted_utilization`. Si tiene otro nombre, solo habría que cambiar `u.weighted_utilization`; el resto de la lógica permanece igual.